

Random maze generation by recursive division, and shortest-path search to the exit

Mohamed Ali Tewfik Hamlil

M1 Artificial Intelligence, Université Grenoble Alpes
Algorithmic Problem Solving — APP1

September 12, 2026

Abstract

A rectangular area of $W \times H$ cells is laid out as a maze by recursive division: a full wall across the longest dimension, exactly one door in it, and the same rule inside each of the two halves. This report states that as a graph problem, shows that the requirements force the maze to be a spanning tree of the grid graph, and derives a generation algorithm together with the data structure it runs on and a proof that what it returns is that spanning tree. Giving every door unit weight makes the length of a route the number of doors crossed, which is what the shortest-path search of Section 4 minimises and what the Manhattan and Euclidean distances are compared against as heuristic guidance. A C++ implementation measures the claims on generated instances, and the closing section gives the cost of both algorithms and places them against the literature.

Contents

1	Problem formulation	3
1.1	Statement	3
1.2	From a drawing of walls to a graph	3
1.3	What the specification forces E to be	4
1.4	Constraints on the exit	5
1.5	The two problems, restated on the graph	5
1.6	Conventions	5
1.7	Scope of the deliverable	5
2	Generating the maze into a graph	6
2.1	What has to be stored	6
2.2	The node record	6
2.3	Regions, named by two corners	7
2.4	The recursion stops at a corridor	7
2.5	One division contributes exactly one door	7
2.6	The region tree	8
2.7	One division step	8
2.8	The algorithm	10
2.9	Why the result is a spanning tree	10
2.10	The graph that comes out	11
3	Encoding the distance between adjacent cells as 1	11
3.1	One door, one step	11
3.2	What the model buys	12
3.3	What the model forbids	12
3.4	Why the shortest route is not in doubt	12
4	Shortest path to the exit, with distance as heuristic guidance	13
4.1	Dijkstra on the maze	13
4.2	The two distances to the exit	13
4.3	Two ways to use an estimate	13
4.4	What tie-breaking can and cannot do	14
4.5	The algorithm	15
5	Testing with a C++ implementation	15
5.1	What was built	15
5.2	What is checked	16
5.3	What it measured	16
5.4	Which estimate to use	16
6	Complexity, and what the literature says	17
6.1	Size of the data structure	17
6.2	Cost of generation	17
6.3	Cost of solving	17
6.4	Why the heuristic barely helps	18
6.5	Where recursive division sits	18

1 Problem formulation

1.1 Statement

A rectangular area of $W \times H$ unit cells is to be laid out as a maze.

Three things are fixed. The layout is drawn **at random**, so W and H do not determine it. The area is **enclosed** and has a single **exit**. It is entered at an arbitrary cell.

Two problems follow. Both are settled on paper first, then analysed for cost.

1. Given the width and the height of the initial area, generate a maze.
2. Find the path from any position in the maze to the exit. The specification states it is unique, hence shortest.

The construction is prescribed.

Divide the allocated space with a full **wall** across its longest dimension. Leave exactly one **door** in that wall. Apply the same rule to each of the two spaces this creates.

Figure 1 is the entire rule. Everything below follows from it.

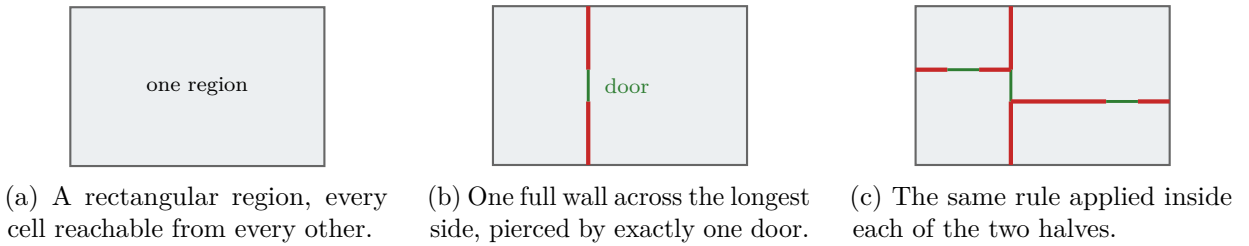


Figure 1: The construction rule. Each application turns one region into two regions of the same kind joined by a single door, which is what makes this a divide-and-conquer problem and the decomposition a binary tree.

What “at random” binds. Half the output, not all of it.

The cells are fixed by W and H . The same WH positions appear in every run, and their coordinates are public the moment the dimensions are.

What must be unpredictable is **which doors were kept** — the edge set.

So the requirement falls on the draws, not on the storage. A rule that takes the wall from the geometry, the midpoint say, makes the maze a function of W and H . Anyone holding those two numbers could rebuild it.

1.2 From a drawing of walls to a graph

A maze on paper is a picture of walls. What an algorithm can generate, store and search is a graph.

That translation is the first decision of this report, and every later section rests on it.

Definition 1 (Area, cells, grid graph). Let $W, H \in \mathbb{N}^*$ be the width and the height of the area, in unit cells. The cells are

$$V = \{(x, y) \in \mathbb{N}^2 : 0 \leq x < W, 0 \leq y < H\}, \quad |V| = WH,$$

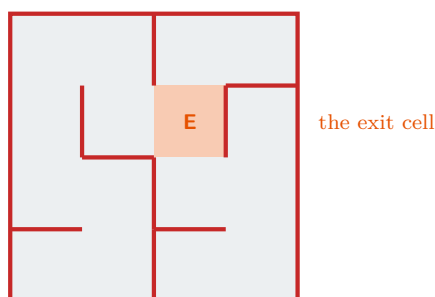
and two cells are *adjacent* when they share a side, that is when $|x - x'| + |y - y'| = 1$. The *grid graph* $G_{\text{grid}} = (V, E_{\text{grid}})$ has the cells as vertices and the adjacent pairs as edges, so

$$|E_{\text{grid}}| = \underbrace{H(W - 1)}_{\text{horizontal}} + \underbrace{W(H - 1)}_{\text{vertical}} = 2WH - W - H.$$

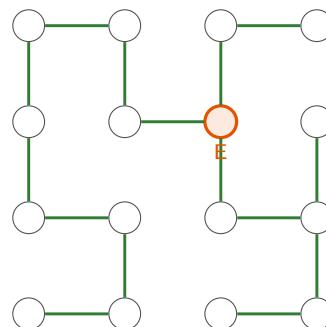
Definition 2 (Maze). A *maze* on that area is a spanning subgraph $M = (V, E)$ with $E \subseteq E_{\text{grid}}$. An edge of E is a **door**: the two cells it joins can be walked between. A pair of $E_{\text{grid}} \setminus E$ is a **wall segment**. One distinguished cell v_{exit} is the exit.

Figure 2 shows the two readings of one object.

The vertex set is fixed by W and H alone. So generating a maze means nothing more than **choosing which subset of E_{grid} to keep**.



(a) The maze as walls. The area is enclosed; the exit is a cell, not a gap in the outer wall.



(b) The same maze as a spanning tree of the grid graph.

Figure 2: A maze is not a picture but a graph. Cells become vertices, doors become edges, and a wall is simply an edge of the grid graph that is absent. The exit is a vertex marked on the finished tree — here an interior one, with degree 3 — and neither its position nor its degree affects anything the construction does.

1.3 What the specification forces E to be

Three requirements follow from the statement.

- (R1) **Every cell reaches the exit.** The entry position is arbitrary, so any cell may be a start and each must have a route out. M is connected.
- (R2) **There is exactly one exit.** One cell is designated v_{exit} . It is a *cell*, not a gap in the outer wall: the area is enclosed on every side, and the exit is the one place leading out of it. It may lie anywhere, including strictly inside.
- (R3) **The route out does not fork.** The path to the exit is required to be unique, hence shortest. A cycle in M would give two distinct simple routes between the cells it passes through, so M is acyclic.

Connected and acyclic on a fixed vertex set is a spanning tree. The shape of the answer is settled before any algorithm is written.

Proposition 1 (The object to produce). *A subgraph $M = (V, E)$ of G_{grid} satisfies (R1) and (R3) if and only if it is a spanning tree of G_{grid} . It then has $|E| = WH - 1$ doors, and between any two cells there is exactly one simple path, which is therefore also a shortest one.*

That last clause is what makes Problem 2 well posed. Asking for *the* path out is only meaningful because there is one.

It is also the lever for the rest of the report: Section 2 produces such a tree **by construction**, rather than by checking afterwards.

1.4 Constraints on the exit

Three questions about v_{exit} , none of which constrains the construction.

Chosen before or after generation?

After. Generation takes W and H and nothing else, and no step of it could consult an exit. By Proposition 1 every cell is reachable from every other, so any cell designated afterwards works.

On the boundary?

No. The exit is a cell of the area, not a hole in the wall enclosing it, so nothing obliges it to touch the perimeter.

Exactly one door?

No. Uniqueness of the route out comes from M being a tree, not from the exit's degree: with 1, 2, 3 or 4 doors there is still exactly one simple path to it from anywhere.

The exit carries no constraint. It is a **label applied to a finished tree**, and every cell is an equally valid candidate. Figure 2 marks an interior one of degree 3.

1.5 The two problems, restated on the graph

Problem 1 — Generation.

Given W , H and a source of random bits, produce a spanning tree M of G_{grid} , drawn so that no two runs need agree, together with the structure recording how it was built. The exit is not an input.

Problem 2 — Escape.

Given M and any start cell $s \in V$, return the unique simple path from s to v_{exit} .

Problem 1 is solved in Section 2.

Problem 2 needs a notion of length first. Section 3 gives every door weight 1; Section 4 runs the shortest-path search that weight makes meaningful, guided by the Manhattan and Euclidean distances.

1.6 Conventions

Symbol	Meaning
W, H	width and height of the area, in cells
$n = WH$	number of cells, the size of the input
(x, y)	cell in column x (left to right), row y (top to bottom)
$G_{\text{grid}}, E_{\text{grid}}$	the grid graph and its edges: every adjacency
$M = (V, E)$	the maze produced: a spanning tree of G_{grid}
v_{exit}	the single exit cell, anywhere in V
$(x_0, y_0)-(x_1, y_1)$	a region, by its upper-left and lower-right cells

1.7 Scope of the deliverable

The specification fixes the contents, not only the topic: a data structure and what it holds, an algorithm per problem, the size of that structure, and the cost of both algorithms.

Required	Where	State
Data structure, and what it stores	Section 2	done
Maze generation algorithm	Section 2	done
Cost of a door, and of a path	Section 3	done
Path-finding algorithm	Section 4	done
Implementation (optional)	Section 5	done
Size of the structure, cost of both algorithms	Section 6	done

2 Generating the maze into a graph

Section 1 fixed the target: a spanning tree of G_{grid} , drawn at random.

This section builds one. The structure comes first, because the algorithm is written against it. The guarantee comes last, because the construction rule is what supplies it.

2.1 What has to be stored

Two different things, and confusing them is the usual way to store neither well.

The maze — which doors exist.

What a walker needs, and all that Problem 2 requires.

The way it was built — the region decomposition.

Which wall went where, in what order. The binary tree the divide-and-conquer rule produces for free, and the object Section 6 measures.

2.2 The node record

The maze is a set of nodes, one per cell. A node holds

$$\text{Node} = \langle \underbrace{\text{value}}_{\text{its } (x,y) \text{ coordinates}}, \underbrace{\text{neighbours}}_{\text{the cells it has a door to}} \rangle.$$

The neighbour list is **empty when the node is created**. A cell has four sides, so it never holds more than four entries, and it grows only when the construction puts a **door** on that node.

An index $\langle P_i, P_j \rangle$ — a $W \times H$ array of references — maps a coordinate pair to its node in $O(1)$. A division reaches the cells it wants without searching. Figure 3 is one node before and after.

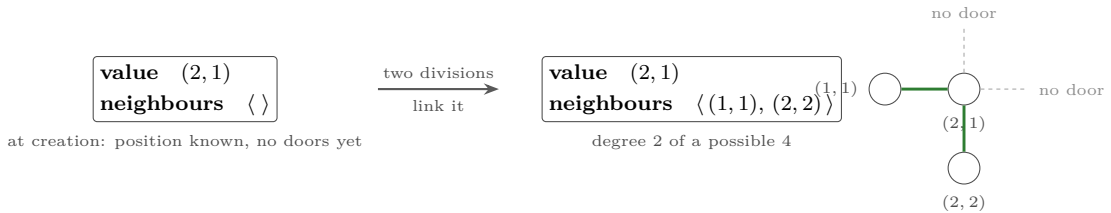


Figure 3: The node record. A node carries its coordinates and a list of the neighbours it has a door to; the list is empty when the node is made and is appended to only by a division whose door lands on that node. A cell has four sides, so the list never exceeds four entries, and in the finished maze the entries across all nodes number $2(WH - 1)$ — each door recorded once at each of its two ends.

Each door is recorded at both ends, so the finished maze holds $2(WH - 1)$ entries for $WH - 1$ doors. That buys enumeration of a cell's exits in $O(\deg)$ from the cell alone, which is what Section 4 does at every step.

2.3 Regions, named by two corners

A region is a rectangle of cells, named by the two cells at its ends: $(x_0, y_0) - (x_1, y_1)$, upper-left and lower-right, with $x_0 \leq x_1$ and $y_0 \leq y_1$. The whole area is $(0, 0) - (W - 1, H - 1)$.

Both corners are **cells of the region**, not fences around it. A region spans $x_1 - x_0 + 1$ columns and $y_1 - y_0 + 1$ rows.

In the pseudocode the two corners are p and q , and a cell's coordinates are reached by index: $p[0]$ is its x , $p[1]$ its y . Indexing rather than naming them lets the division below be written once for both axes.

2.4 The recursion stops at a corridor

Definition 3 (Corridor). A region is a *corridor* when one of its two sides is a single cell: $x_0 = x_1$, one column wide and any height, or $y_0 = y_1$, one row tall and any width. A single cell, where both hold, is the smallest corridor.

A corridor is the case where the construction has nothing to decide.

Its cells lie in a line, so the subgraph G_{grid} induces on it is already a path. A path on m vertices has $m - 1$ edges and is a tree, so it is its own and only spanning tree.

Put a **wall** across a corridor and it has exactly one crossing, which is then forced to be the **door**. The division would change nothing.

So the recursion stops there. **SpanCorridor** lays those forced doors and returns the leaf recording them.

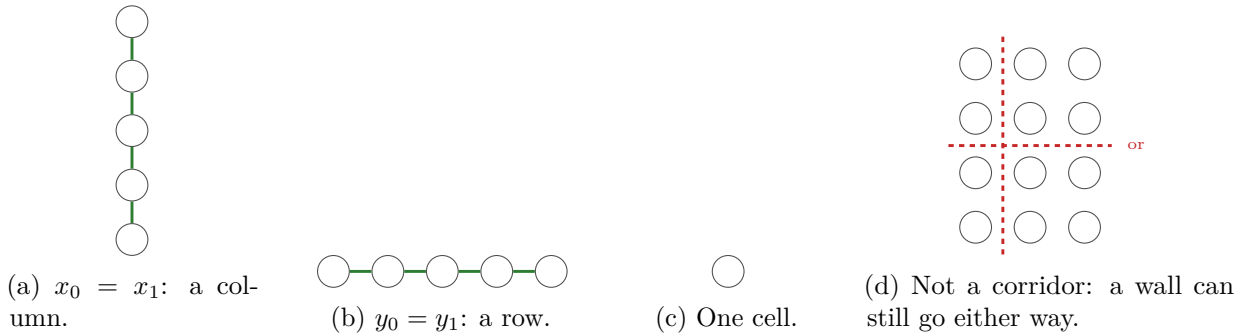


Figure 4: A *corridor* is a region one cell thick, in either direction. Its cells lie in a line, so the induced graph is already a path and there is exactly one way to span it — nothing about a corridor is left to decide, which is why the recursion stops at one. A region thicker than that, on the right, still has both a choice of orientation and a choice of position.

Stopping at a corridor means **every region reaching a division has both sides at least 2**. Both axes always admit a wall, so the orientation is a plain coin with no test in front of it.

2.5 One division contributes exactly one door

Take any door $e = (u, v)$ of the finished maze and walk down the region tree from the root.

At each internal node, u and v either fall in the same child — and the walk continues — or they are separated. They are separated exactly when e crosses that node's wall, and then e is that node's door, being the only crossing there is.

Otherwise the walk ends at a leaf holding both, and e is one of the doors that leaf laid along its corridor.

So the doors split cleanly in two: one per internal node, plus the interiors of the corridor leaves. Nothing is counted twice, nothing is missed.

That licenses the construction to run **forwards**. A division adds its own door and nothing else; a leaf adds its corridor. A wall is never represented at all — it is simply the doors that were never added.

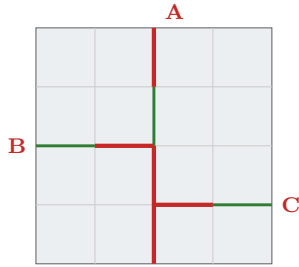
2.6 The region tree

The recursion's shape is a binary tree, one node per call.

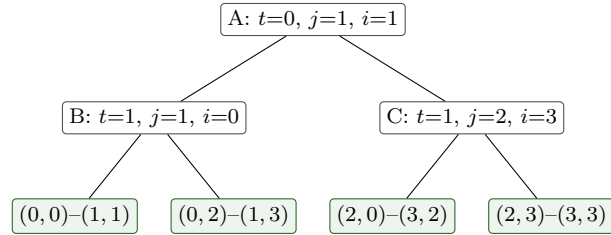
- An **internal node** records one division: its orientation t , the boundary j the wall sits on, the position i of its door. Its two children are the two regions the division created.
- A **leaf** is a corridor, of one cell or many.

A binary tree with L leaves has $L - 1$ internal nodes. So of the $WH - 1$ doors, $L - 1$ are division doors and $WH - L$ are corridor doors.

Figure 5 shows the first three divisions of a 4×4 area beside the top of the tree they build.



(a) The first three divisions of a 4×4 area.



(b) The top of the region tree it builds. The four boxes at the bottom are regions still to be divided, not corridors.

Figure 5: Every division is one internal node. The node records the orientation t , the wall position j and the door position i it drew, and its two children are the two regions it created; a leaf is a corridor. A tree with L leaves has $L - 1$ internal nodes, so of the $WH - 1$ doors of Proposition 1, $L - 1$ are division doors and the rest are laid down inside the corridors.

2.7 One division step

A call handles a region $(x_0, y_0)-(x_1, y_1)$. If it is a corridor, Section 2.4 disposes of it. Otherwise both sides are at least 2, and:

1. **Toss for the axis t to cut.** Both admit a wall, so the coin is free. Write $t = 0$ for the x axis and $t = 1$ for the y axis; then $1 - t$ is the other one.
2. **Draw the wall position j on axis t ,** uniformly in $[p[t], q[t] - 1]$, **and the door position i on the other axis,** uniformly in $[p[1 - t], q[1 - t]]$.
3. **Link the two cells the door falls on,** appending each to the other's neighbour list.

4. **Recurse** on the two regions — the same corners, cut at j on axis t — and hang them under the node recording $\langle t, j, i \rangle$.

Remark (Departure from the specification). The specification puts the wall across the *longest* dimension. This report tosses for it instead. Correctness is untouched: Proposition 2 never uses which axis was cut, only that one was.



(a) The call links one pair and stops. Every other node is still isolated; no crossing was examined, let alone removed.

(b) After both recursive calls return. Each half is a tree, and the door is still the only edge between them.

Figure 6: One division, with $t = 0$ — the coin named the x axis, so the wall is vertical. The call draws the wall position j on that axis and the door position i on the other, links the two cells the door falls on, and recurses on the two halves. Counting edges: 11 on the left, 7 on the right, one door, total $19 = 20 - 1$ on 20 cells — the tally of Proposition 2.

2.8 The algorithm

Algorithm 1: Maze generation by recursive division.

Input: $W, H \geq 1$, the width and height of the area in cells. The bound is a precondition, not a convention: at $W = 0$ the region $(0, 0) - (-1, H - 1)$ holds no cell and there is nothing for the first draw to land on.

Output: N , the set of nodes holding a spanning tree of G_{grid} ; r , the root of the region tree that built it.

```

1 Function GenerateMaze( $W, H$ ):
2    $N \leftarrow \{ \text{NewNode}(x, y) : 0 \leq x < W, 0 \leq y < H \}$ ;    // every neighbour list empty
3    $r \leftarrow \text{Divide}(N, (0, 0), (W - 1, H - 1))$ ;
4   return ( $N, r$ );

5 Function Divide( $N, p, q$ ):
6   if  $p[0] = q[0]$  or  $p[1] = q[1]$  then return SpanCorridor( $N, p, q$ );
7    $t \leftarrow \text{RandomBit}()$ ;                                // the axis to cut: 0 is  $x$ , 1 is  $y$ 
8    $j \leftarrow \text{RandomIn}(p[t], q[t] - 1)$ ;                  // wall between  $j$  and  $j + 1$  on axis  $t$ 
9    $i \leftarrow \text{RandomIn}(p[1-t], q[1-t])$ ;                  // the one door, at  $i$  on the other axis
10   $\text{Link}(N, \text{CellAt}(t, j, i), \text{CellAt}(t, j + 1, i))$ ;
11   $\ell \leftarrow \text{Divide}(N, p, \text{WithAxis}(q, t, j))$ ;
12   $r \leftarrow \text{Divide}(N, \text{WithAxis}(p, t, j + 1), q)$ ;
13  return CutNode( $t, j, i, \ell, r$ );

14 Function SpanCorridor( $N, p, q$ ):
15  for  $x \leftarrow p[0]$  to  $q[0] - 1$  do  $\text{Link}(N, (x, p[1]), (x + 1, p[1]))$ ;
16  for  $y \leftarrow p[1]$  to  $q[1] - 1$  do  $\text{Link}(N, (p[0], y), (p[0], y + 1))$ ;
17  return Leaf( $p, q$ );

18 Function Link( $N, u, v$ ):
19   $\text{append } N[v] \text{ to } N[u].\text{neighbours}$ ;
20   $\text{append } N[u] \text{ to } N[v].\text{neighbours}$ ;

21 Function CellAt( $t, u, v$ ):
22  return the cell whose axis- $t$  coordinate is  $u$  and whose other is  $v$ ;

23 Function WithAxis( $p, t, u$ ):
24  return the cell  $p$  with its axis- $t$  coordinate replaced by  $u$ ;
```

`SpanCorridor()` is the whole base case. It lays the corridor's forced doors *and* returns the leaf recording the region.

It needs no case analysis: on a corridor one of its loops is empty by construction, on a single cell both are. The same two loops cover a row, a column and one cell.

Every `Link()` joins two cells of the region that called it, and they are adjacent. So every appended entry is a genuine edge of G_{grid} .

The exit plays no part, for the reasons in Section 1.4. The algorithm never reads it, and any cell may be designated once the tree exists.

2.9 Why the result is a spanning tree

Proposition 2 (Correctness of Algorithm 1). *On a region of m cells, Divide leaves the subgraph induced on that region connected and acyclic, with exactly $m - 1$ doors. In particular GenerateMaze returns a spanning tree of G_{grid} .*

Proof. By induction on m .

Base. If the region has a side of one it is a corridor of m cells in a line, and **SpanCorridor** joins each to the next: a path, connected and acyclic, carrying $m - 1$ doors. This covers $m = 1$, where both loops are empty and $m - 1 = 0$.

Step. Otherwise both sides are at least 2, so $q[0] - p[0] \geq 1$ and $q[1] - p[1] \geq 1$. Whichever axis the coin names, $q[t] - p[t] \geq 1$, so j is drawn from a non-empty range and the two sub-regions R_ℓ and R_r are non-empty, with $m_\ell + m_r = m$.

The call adds the single door at i , whose two cells lie one in each sub-region, then recurses. By the induction hypothesis the two calls leave their regions connected and acyclic with $m_\ell - 1$ and $m_r - 1$ doors. Neither can add an edge between the sub-regions, since every **Link** a call makes joins two cells of its own region.

So the door at i is the only edge between R_ℓ and R_r . Joining two disjoint trees by a single edge yields a tree. It is connected, since any two cells are joined inside their own half or by crossing that door. It is acyclic, since a cycle would have to cross the wall an even, hence non-zero, number of times while only one crossing exists. The door count is $(m_\ell - 1) + (m_r - 1) + 1 = m - 1$.

Applied to the whole area this gives a connected acyclic spanning subgraph of G_{grid} on all WH cells: a spanning tree, with $WH - 1$ doors. $\square$

The guarantee is structural. Nothing is checked after the fact and no attempt is ever rejected and retried.

The uniqueness of the route out — the whole of Problem 2 — follows from the shape of the recursion, not from a property that had to be tested.

2.10 The graph that comes out

Generation returns the set N of nodes, each carrying its coordinates and the doors it ended up with, plus the root r of the region tree.

Three readings of that one result, none of them a separate copy:

View	Held as	Used by
Adjacency	each node's neighbour list	the search of Section 4
Position	each node's <i>value</i>	the heuristics of Section 4
How it was built	the region tree rooted at r	the cost analysis of Section 6

The second row is what makes Section 4 possible. A heuristic estimating the remaining distance to the exit needs the coordinates of the cell it stands on, and they are in the node.

Figure 2b is the output of one 4×4 run, whose first three divisions are drawn in Figure 5a: 15 doors on 16 cells, one path between any two of them.

Section 3 adds a length to those doors, so that “the path out” becomes a quantity a search can minimise.

3 Encoding the distance between adjacent cells as 1

3.1 One door, one step

Section 2 produced a graph. It has no lengths on it yet, so “the shortest way out” is not a question it can answer.

The cost model is the simplest one the problem admits.

Walking from a room to an adjacent room through a **door** is **one step**. Every edge of M carries weight $w(e) = 1$.

The length of a route is then the **number of doors it crosses**, and nothing else. A route through $k + 1$ cells costs k .

Definition 4 (Distance in the maze). For cells u, v , let $d(u, v)$ be the least total weight of a route from u to v in M . Under unit weights this is the number of doors on that route.

3.2 What the model buys

Three things follow immediately, and together they are why the next section can be written at all.

The weights are non-negative.

That is the one precondition Dijkstra’s algorithm asks of a graph [7]. So the search of Section 4 applies to M without further argument.

Every cell is a vertex; every door is an edge.

The maze is already the graph the search runs on. No transformation, no auxiliary structure: the node records of Section 2 are the adjacency list.

Distance is depth.

With all weights equal, $d(s, \cdot)$ counts doors, so the cells at distance k from s form a layer. This is the observation Section 4 turns on.

3.3 What the model forbids

Stating the model’s limits is part of stating the model.

- **No weighted doors.** A door that is slower to pass — a staircase, a flooded corridor — cannot be expressed. The graph would stay the same and only w would change, so the search would still work; but every count in Section 6 is a count of *doors*, and would stop meaning *time*.
- **No diagonal movement.** Cells are adjacent only across a shared side, so the grid is 4-connected. This is what makes the Manhattan distance the natural estimate in Section 4; on an 8-connected grid the right estimate would be the octile distance instead.
- **No cost for turning.** A route that zigzags and a route that runs straight cost the same if they cross the same number of doors.

3.4 Why the shortest route is not in doubt

On a general graph, a unit-weight shortest path is one of possibly many. Here it is not.

By Proposition 1 the maze is a spanning tree, so between any two cells there is exactly one simple route. That route is therefore trivially the shortest, whatever weights are used.

The search of Section 4 is not choosing between routes. There is only one. What it does is **locate** it, and the only thing that separates one search from another is **how much of the maze it looks at on the way**.

That is the measure Section 5 reports and Section 6 explains: not the length of the answer, which is fixed, but the number of cells expanded before the answer is reached.

4 Shortest path to the exit, with distance as heuristic guidance

4.1 Dijkstra on the maze

Every door weighs 1 and no weight is negative, so Dijkstra’s algorithm [7] applies to M directly.

It keeps a distance $g(v)$ for every cell, a priority queue of cells reached but not yet settled, and repeatedly settles the nearest one, relaxing its doors.

Two details matter for what follows.

It stops when the exit is *settled*, not when it is reached.

The exit is put in the queue like any other cell; the search ends when it comes back out. Stopping on first sight would be a different algorithm with a different count [11].

On unit weights it is breadth-first search.

With all weights equal, the cell of least g is always one of least depth, so cells are settled in non-decreasing depth order — the layers of Section 3. Dijkstra generalises breadth-first search to weighted graphs [4, Ch. 22]; here the generalisation buys nothing but a log factor.

4.2 The two distances to the exit

The search has something Dijkstra does not use: every node carries its coordinates, so the *straight-line* remoteness of a cell from the exit is available for free.

Definition 5 (Estimates). For a cell $v = (x, y)$ and the exit $v_{\text{exit}} = (x_e, y_e)$,

$$h_M(v) = |x - x_e| + |y - y_e|, \quad h_E(v) = \sqrt{(x - x_e)^2 + (y - y_e)^2}.$$

Both are **admissible**: neither can exceed the true number of doors to the exit. Any route changes one coordinate by one per step, so it crosses at least h_M doors on an empty grid, and walls only remove edges, which can only lengthen it. Then $h_E \leq h_M$ by the inequality between the Euclidean and the Manhattan norm.

$h_M \geq h_E$ everywhere, with equality only when the exit shares a row or a column with the cell. **Manhattan dominates Euclidean**: it is the exact distance on a 4-connected empty grid, while the Euclidean value is a straight line no walker can take.

4.3 Two ways to use an estimate

The estimate can enter the search at two different strengths, and the report measures both because they behave very differently.

As a tie-break.

Order the queue by (g, h) . The primary key stays the distance already travelled; h decides only between cells the search has reached at *equal* cost. No computed distance changes.

As a priority.

Order the queue by $(g + h, h)$. This is A* [10]: the estimate is added to the cost, so the search leans towards the exit instead of spreading evenly.

A* does **not** trade the shortest route for a quicker answer. With an admissible estimate it returns an optimal route, exactly as Dijkstra does; it simply settles fewer cells on the way [10]. Indeed Dijkstra *is* A* with $h \equiv 0$ — one algorithm, told nothing.

The methods that do trade optimality for speed are different ones, and are not used here. **Greedy best-first search** orders by h alone, discarding g : fast, and neither optimal nor reliably complete. **Weighted A*** orders by $g + w h$ with $w > 1$, which returns a route at most w times longer than the shortest. Both are worth naming because A* is often remembered as one of them.

4.4 What tie-breaking can and cannot do

The tie-break is the weaker of the two, and by how much can be settled before measuring anything.

Proposition 3 (Ceiling on the tie-break). *Let $d = g(v_{\text{exit}})$. Under any tie-break, the search settles every cell with $g < d$, and then some subset of the cells with $g = d$. Ordering ties by h settles none of that last layer before the exit.*

Proof. The primary key is g , so cells leave the queue in non-decreasing g . Every cell with $g < d$ therefore leaves before v_{exit} , whatever the tie-break: the set $\{v : g(v) < d\}$ is expanded in all cases. The only freedom is the order inside the layer $g = d$. Both estimates are non-negative and vanish exactly at the exit, so $h(v_{\text{exit}}) = 0$ is strictly least in that layer, and the exit is taken first. $\square$

A tie-break cannot prune anything. Its entire effect is bounded by one layer, and the cells with $g < d$ are **necessarily expanded** by any correct search of this shape [6]. It follows too that h_M and h_E must behave **identically** here: both rank the exit first and the search ends there.

Ordering ties by h is the standard $[f, h]$ strategy of the heuristic-search literature [5]; what is unusual here is only that the primary key is blind, so the ceiling above is a corollary of the necessarily-expanded result rather than a new claim.

Using the estimate as a *priority* escapes that ceiling, because it changes f and so changes which cells fall below the threshold at all. A cell deep in a dead end is dropped as soon as its depth plus its estimate exceeds d .

4.5 The algorithm

Algorithm 2: Shortest route to the exit, with an estimate as guide.

Input: the maze M , a start cell s , the exit v_{exit} , an estimate h (possibly $h \equiv 0$), and whether it is used as a tie-break or as a priority.

Output: the route from s to v_{exit} , and the number of cells settled.

```

1 Function ShortestPathToExit( $M, s, v_{\text{exit}}, h, \text{use}$ ):
2    $g[v] \leftarrow \infty$  for every cell;  $g[s] \leftarrow 0$ ;
3    $\text{settled}[v] \leftarrow \text{FALSE}$  for every cell;
4    $Q \leftarrow$  empty priority queue;  $\text{Push}(Q, s)$ ;
5    $\text{expanded} \leftarrow 0$ ;
6   while  $Q$  is not empty do
7      $u \leftarrow \text{PopLeast}(Q)$ ; // least  $(g + h, h)$ , or least  $(g, h)$ 
8     if  $\text{settled}[u]$  then continue;
9      $\text{settled}[u] \leftarrow \text{TRUE}$ ;  $\text{expanded} \leftarrow \text{expanded} + 1$ ;
10    if  $u = v_{\text{exit}}$  then break;
11    for  $v \in M.\text{neighbours}(u)$  do
12      if  $g[u] + 1 < g[v]$  then
13         $g[v] \leftarrow g[u] + 1$ ; // one door, one step
14         $\text{parent}[v] \leftarrow u$ ;  $\text{Push}(Q, v)$ ;
15  return  $\text{RouteBackFrom}(\text{parent}, v_{\text{exit}})$ ,  $\text{expanded}$ ;
```

The queue is ordered by $g + h$ when the estimate is a priority and by g when it is a tie-break, with h separating equals in both cases. Setting $h \equiv 0$ recovers plain Dijkstra, so one procedure covers every row of the table in Section 5.

expanded is the figure that section reports. Every setting returns the same route — Proposition 1 leaves only one — so it is the only thing there is to compare.

5 Testing with a C++ implementation

The handout leaves implementation optional. It is done here because two claims in this report are the kind that ought to be measured rather than asserted: that generation always returns a spanning tree, and that a heuristic guide changes how much of the maze a search looks at.

5.1 What was built

The code is C++20, built with CMake, compiled with `-Wall -Wextra -Wpedantic -Werror`, and lives beside this report in `implementation/`.

File	Role
<code>cell.hpp</code>	a cell, and the two axis operations of Section 2.3
<code>maze.{hpp,cpp}</code>	the node records: coordinates and neighbour lists
<code>maze_generator.{hpp,cpp}</code>	Algorithm 1, and the region tree
<code>maze_solver.{hpp,cpp}</code>	Algorithm 2, with the estimate as tie-break or priority
<code>maze_invariants.{hpp,cpp}</code>	the properties a finished maze must have, checked from outside
<code>maze_svg.{hpp,cpp}</code>	drawing, for looking at a result

The invariants are a separate module on purpose: a generator that checked its own work would only ever confirm its own assumptions.

5.2 What is checked

Generation returns a spanning tree.

$WH - 1$ doors, connected, no cell with more than four. Over ten shapes $\times$ twenty seeds, including 1×1 , 1×40 and 40×1 .

A corridor is laid down whole.

A 1×40 area produces a region tree of exactly one leaf and 39 doors, which is Section 2.4 observed rather than argued.

The same seed gives the same maze.

Required for any measurement below to be reproducible.

Every setting returns the same route.

Plain Dijkstra, both estimates as tie-breaks, and both as priorities agree on the path, cell for cell — which is Proposition 1 made visible.

The route is walkable.

Consecutive cells on it are joined by a door.

Bad input is refused.

A zero dimension, a door between cells that do not touch, a door opened twice, a start outside the grid.

5.3 What it measured

Two hundred mazes per size, start at $(0, 0)$, exit at the far corner, counting the cells each search settled before the exit came out of the queue.

Size	Cells	Steps	estimate as tie-break			as priority (A^*)	
			none	Manhattan	Euclidean	Manhattan	Euclidean
8×8	64	22.6	55.6	54.8	54.8	41.0	45.8
16×16	256	64.4	219.1	217.7	217.7	172.1	186.9
32×32	1024	188.0	879.5	877.2	877.2	754.9	789.7
64×64	4096	549.4	3587.1	3584.1	3584.1	3278.8	3359.0
128×128	16384	1595.9	14128.5	14124.0	14124.0	13303.5	13488.9

Every one of the 1000 mazes generated for the table was checked to be a spanning tree. All of them were.

5.4 Which estimate to use

The test was meant to settle Manhattan against Euclidean. It settles rather more than that.

As a tie-break, neither.

The saving is 4.5 cells in 14 128, or 0.03%, and the two estimates are *exactly* equal in every row. That is Proposition 3 arriving as predicted: the tie-break can only reorder the last layer, and both estimates put the exit first in it.

As a priority, Manhattan.

It expands fewer cells than Euclidean at every size — 13 303 against 13 489 at 128×128 — which is the dominance of Section 4 showing up as a measurement.

But the gain is small either way.

A^* with Manhattan still settles 81% of the maze, against 86% for plain Dijkstra. The search

looks at most of the maze whatever it is told.

The answer is **Manhattan, used as a priority** — and the more useful finding is how little any of it buys. Section 6 says why that is a property of mazes and not of this implementation.

6 Complexity, and what the literature says

6.1 Size of the data structure

Write $n = WH$ for the number of cells.

What	How many	Why
Nodes	n	one per cell
Neighbour entries	$2(n - 1)$	$n - 1$ doors, each recorded at both ends
Index $\langle P_i, P_j \rangle$	n	one reference per cell
Region tree nodes	$2L - 1$	L corridor leaves, $L - 1$ divisions

Everything is $\Theta(n)$. The grid graph is sparse — $|E_{\text{grid}}| = 2n - W - H$ — so no representation of it can be smaller than linear, and the maze itself is sparser still.

6.2 Cost of generation

Each call of **Divide** does $O(1)$ work of its own: two draws, one **Link**, and the recursion. Each call of **SpanCorridor** does work proportional to the length of its corridor.

Every door is opened exactly once, by one division or by one corridor (Section 2.5), and there are $n - 1$ of them. The region tree has $2L - 1$ nodes with $L \leq n$. So

Generation runs in $\Theta(n) = \Theta(WH)$ **time**, which is optimal: any generator must write $n - 1$ doors.

The recursion is not balanced. The wall position is uniform, not central, so a division can leave a single column on one side — the same skew that makes quicksort’s worst case quadratic. Depth is $O(n)$ in the worst case and $O(\log n)$ in expectation, and the constant is visible:

Area	Corridor leaves	Mean depth	Depth if balanced
16×16	55.2	11.6	5.8
64×64	671.7	21.2	9.4
128×128	2388.5	26.7	11.2

Roughly 2.4 times the depth of a balanced tree on the same number of leaves, at every size measured. That is the cost of drawing the wall uniformly, and it is the figure a stack-depth bound has to be written against.

Stopping at corridors changes no asymptotics and a great deal of constant: 1301 calls instead of 8191 on 64×64 , and one call instead of 79 on 1×40 .

6.3 Cost of solving

The maze is a tree, so $|E| = n - 1$ and the graph the search runs on is as sparse as a connected graph can be. With a binary heap, Dijkstra costs $O((|V| + |E|) \log |V|) = O(n \log n)$ [4, Ch. 22].

That log buys nothing here. All weights are 1, so Dijkstra settles cells in depth order and is breadth-first search wearing a priority queue; plain breadth-first search would do the same work

in $\Theta(n)$. The queue is kept because the report’s algorithm is written to accept an estimate, and A* needs it.

6.4 Why the heuristic barely helps

The measurements of Section 5 are not a disappointing implementation. They are what the search literature predicts for mazes, for two separate reasons that are worth keeping apart.

A tie-break cannot prune.

Every cell nearer the start than the exit is *necessarily expanded* by any admissible search with this priority function [6], so only the last layer is negotiable. Proposition 3 is that result applied here, and the measured 0.03% is it.

A* prunes subtrees, not rival routes.

On a general grid much of A*’s advantage is in discarding the many equal-length routes between the same pair of cells. A perfect maze has *none*: it is a tree, so there are no rival routes to discard. What is left is pruning dead-end branches whose depth plus estimate already exceeds d — real, but a smaller effect.

The estimate is very inaccurate on mazes.

This is the measured result in the literature, not an intuition. Sturtevant et al. [14] report that on 512×512 maze maps the geometric estimate averages 151 where the true distance is 512–768, and that a memory-based heuristic expands over eleven times fewer nodes. A straight line between two cells says little about a route that must wind.

The walls, not the algorithm, are what defeats the guide. The estimate ignores them, and in a maze they are the whole problem.

A consequence worth stating for completeness: the grid-pathfinding techniques that give the largest speedups — Jump Point Search and the symmetry-breaking family [9] — work by collapsing equal-cost alternative routes. A perfect maze has no path symmetry at all, so that line of work has nothing to prune here. Standard benchmark sets include maze maps for exactly this reason [13].

6.5 Where recursive division sits

The maze-generation literature is small, and mostly practitioner-written; the distinction is marked below because it matters for how much weight each claim carries.

The vocabulary. What this report builds is a **perfect maze** — a maze in which any two cells are joined by a unique path — which is the same object as a spanning tree of the grid graph. The term is Pullen’s [12] and is used verbatim in peer-reviewed work [2]. Recursive division is a **wall adder**: it places walls, where most generators carve passages. Bellot et al. call it the only wall adder among the standard algorithms.

The alternatives. Randomised depth-first search, randomised Kruskal and Prim, Eller’s algorithm, Hunt-and-Kill, binary tree and sidewinder all produce spanning trees of the same grid, in $\Theta(n)$ or close to it. Two do something this one does not: Aldous–Broder [1, 3] and Wilson’s algorithm [15] sample a **uniform** spanning tree — every perfect maze equally likely. Wilson’s runs in expected mean-hitting time rather than cover time, which on a grid is a $\Theta(\log n)$ improvement over Aldous–Broder.

Recursive division is not uniform, and not nearly. This is the honest position of the algorithm chosen here. Its first division leaves exactly one door crossing a full straight line, and

recursively so; any spanning tree admitting no such decomposition can *never* be produced. The support is a strict subset of the spanning trees, so the distribution is not uniform even before asking how it is weighted within its support. Pullen [12] classifies it accordingly, and records its visible signature: long straight walls crossing the interior, rectangular rooms, a self-similar texture, and a notably direct solution path. The count makes the gap concrete — an n -cell grid has on the order of 3.21^n spanning trees, and recursive division reaches only those with a guillotine decomposition.

Recursive division is chosen for being **simple, linear and recursive**, which is what this APP is about. It is not chosen for the quality of its randomness, and it should not be claimed for it.

What is missing from the literature. No peer-reviewed analysis of the distribution recursive division induces appears to exist. Bellot et al. [2] include it but score only difficulty; Gabrovšek [8] measures six generators and omits it. The characterisation of its bias is practitioner-level throughout. Measuring its degree distribution and straight-run lengths against the uniform spanning tree would close that gap, and is the natural continuation of this work.

References

- [1] David J. Aldous. The random walk construction of uniform spanning trees and uniform labelled trees. *SIAM Journal on Discrete Mathematics*, 3(4):450–465, 1990.
- [2] Valérie Bellot, Maxime Cautrès, Jean-Marie Favreau, Manon Gonzalez-Thauvin, Pascal Lafourcade, Kergann Le Cornec, Maëlig Mosnier, and Sol Rivière-Wekstein. How to generate perfect mazes? *Information Sciences*, 572:444–459, 2021.
- [3] Andrei Z. Broder. Generating random spanning trees. In *30th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 442–447, 1989.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.
- [5] Augusto B. Corrêa, André G. Pereira, and Marcus Ritt. Analyzing tie-breaking strategies for the A* algorithm. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI)*, pages 4715–4721, 2018.
- [6] Rina Dechter and Judea Pearl. Generalized best-first search strategies and the optimality of A*. *Journal of the ACM*, 32(3):505–536, 1985.
- [7] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [8] Peter Gabrovšek. Analysis of maze generating algorithms. *IPSI Transactions on Internet Research*, 15(1):23–30, 2019.
- [9] Daniel Harabor and Alban Grastien. Online graph pruning for pathfinding on grid maps. In *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence*, volume 25, pages 1114–1119, 2011.
- [10] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, SSC-4(2):100–107, 1968.
- [11] Robert C. Holte. Common misconceptions concerning heuristic search. In *Proceedings of the Third Annual Symposium on Combinatorial Search (SoCS)*, pages 46–51, 2010.
- [12] Walter D. Pullen. Think labyrinth: Maze algorithms. <https://www.astrolog.org/labyrinth/algrithm.htm>, 2026. Practitioner reference, not peer-reviewed; the source of the field’s vocabulary and its standard comparison table.
- [13] Nathan R. Sturtevant. Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):144–148, 2012.
- [14] Nathan R. Sturtevant, Ariel Felner, Max Barrer, Jonathan Schaeffer, and Neil Burch. Memory-based heuristics for explicit state spaces. In *Proceedings of the Twenty-First International Joint Conference on Artificial Intelligence (IJCAI)*, pages 609–614, 2009.
- [15] David Bruce Wilson. Generating random spanning trees more quickly than the cover time. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC)*, pages 296–303, 1996.